

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении лабораторной работы №1
по дисциплине «Теория алгоритмов»
«Реализация двумерного клеточного автомата»
Вариант 9

Выполнила студентка группы
№5130201/30101
Проверил

Лаишевкина А.М. _____
Востров А.В. _____

«_____» _____ 2025г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Математическое описание	4
1.1 Клеточный автомат	4
1.2 Функция	4
1.3 Граничные условия	6
1.4 Динамическое поведение	7
2 Особенности реализации	9
2.1 Класс NeumannCellularAutomaton	9
2.1.1 Метод конвертации правила	9
2.1.2 Метод для определения нового состояния ячейки . .	10
2.1.3 Метод, реализующий изменение состояния клетки .	10
2.1.4 Метод, реализующий инициализацию клеток поля случайными значениями	11
2.1.5 Метод, реализующий очистку поля	11
2.1.6 Метод, реализующий изменение правила	11
2.1.7 Метод, реализующий одну итерацию эволюции ав- томата	12
2.2 Визуальное отображение	12
3 Анализ клеточного автомата	15
4 Результаты работы программы	16
Заключение	21
Список литературы	22
Приложение А. Исходный код программы	23

Введение

Цель данной работы - реализация двумерного клеточного автомата с окрестностью фон Неймана и торроидальными граничными условиями. Функция изменения состояния клеток задается следующей формулой:

$$s = \text{№ варианта} * 11 * \text{год рождения} * \text{день} * \text{месяц} = 14887125_{10}$$

Ширина поля и количество итераций должны определяться пользователем. Он также должен иметь возможность выбрать: вручную задать состояние клеток поля или случайным образом. Необходимо проанализировать получившийся клеточный автомат: его поведение, паттерны, «сходимость».

1 Математическое описание

1.1 Клеточный автомат

Клеточный автомат – это дискретная математическая модель, состоящая из регулярной сетки ячеек, каждая из которых может находиться в одном из конечного числа состояний. Эволюция системы происходит пошагово, где состояние каждой ячейки на следующем шаге определяется её текущим состоянием и состояниями ячеек в её окрестности согласно правилу перехода.

Клеточный автомат можно формально определить как:

$$A = (L, S, \mathcal{N}, f)$$

где L – решетка, S – множество состояний, $\mathcal{N}$ – окрестность, f – правило перехода.

Элементы клеточного автомата:

1. Решетка клеток представляет собой конечную двумерную решётку размером $N \times N$ с торроидальными граничными условиями.
2. Состояния клеток и правила перехода определяют динамику системы. Каждая клетка может находиться в одном из конечного множества состояний. Правила перехода определяют следующее состояние клетки. Новое состояние клетки вычисляется на основе конфигурации ее окрестности, причем все обновления применяются одновременно ко всей решетке после завершения расчетов для всех клеток.
3. Начальное состояние – это отображение $s_0 : L \rightarrow S$, задающее конфигурацию на момент $t = 0$.
4. Окрестность фон Неймана (Рис. 1) определяет локальное взаимодействие между клетками. В данной работе рассматривается окрестность, включающая центральную клетку и четырех ее ближайших соседей, имеющих с ней общую сторону.

1.2 Функция

Функция перехода является центральным элементом клеточного автомата и определяет его динамику. В общем виде функция перехода может быть представлена как отображение $f : S^N \rightarrow S$, где S – множество возможных состояний клетки, а N – количество клеток в окрестности.

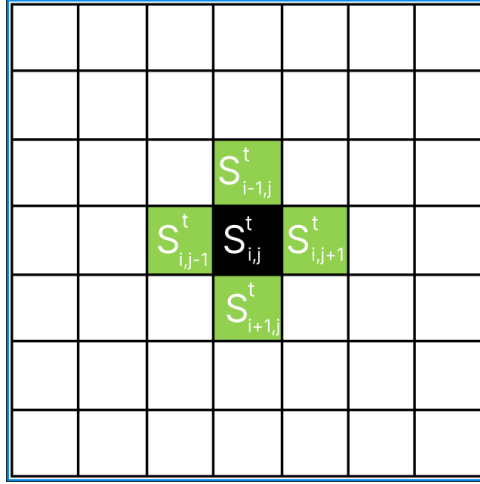


Рис. 1. Окрестность фон Неймана

Для клеточного автомата с окрестностью фон Неймана функция перехода записывается как:

$$s_{i,j}^{t+1} = f(s_{i,j}^t, s_{i-1,j}^t, s_{i+1,j}^t, s_{i,j-1}^t, s_{i,j+1}^t)$$

где $s_{i,j}^t$ – состояние клетки с координатами (i, j) в момент времени t , а функция f определяет новое состояние на основе текущих состояний центральной клетки и её соседей. Для поля размером $N \times N, i, j \in [0, N-1]$

Вектор значений функции, используемой в данной лабораторной работе вычисляется по следующей формуле:

$$s = \mathbb{N}^\circ \text{ варианта} * 11 * \text{год рождения} * \text{день} * \text{месяц} = 14887125_{10} = 111000110010100011010101_2$$

Таблица истинности функции f представлена в [таблице 1](#).

Таблица 1. Функция 14 887 125

$s_{i,j}$	$s_{i-1,j}$	$s_{i+1,j}$	$s_{i,j-1}$	$s_{i,j+1}$	$f(s_{i,j}, s_{i-1,j}, s_{i+1,j}, s_{i,j-1}, s_{i,j+1})$
0	0	0	0	0	0
0	0	0	0	1	0
0	0	0	1	0	0
0	0	0	1	1	1
0	0	1	0	0	1
0	0	1	0	1	1
0	0	1	1	0	1

0	0	1	1	1	0
0	1	0	0	0	0
0	1	0	0	1	1
0	1	0	1	0	0
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	0	1	1
0	1	1	1	0	1
0	1	1	1	1	1
1	0	0	0	0	0
1	0	0	0	1	0
1	0	0	1	0	1
1	0	0	1	1	0
1	0	1	0	0	0
1	0	1	0	1	1
1	0	1	1	0	1
1	0	1	1	1	0
1	1	0	0	0	1
1	1	0	0	1	0
1	1	0	1	0	1
1	1	0	1	1	0
1	1	1	0	0	1
1	1	1	0	1	0
1	1	1	1	0	1
1	1	1	0	1	0
1	1	1	1	0	1
1	1	1	1	1	1

1.3 Граничные условия

В данной работе рассматриваются торроидальные граничные условия. Таким образом, противоположные границы решетки считаются соединенными. Данный подход преобразует двумерное поле в поверхность тора, где верхний и нижний края, а также левый и правый края попарно отождествляются.

Математическая запись для доступа к клеткам при торроидальных условиях имеет вид:

$$s_{i,j} = s_{i \bmod N, j \bmod N}$$

Рассмотрим случаи, затрагивающие граничные условия (где N – размер поля):

- Верхний ряд ($i = 0$):

$$s_{0,j}^{t+1} = f(s_{0,j}^t, s_{N-1,j}^t, s_{1,j}^t, s_{0,j-1}^t, s_{0,j+1}^t)$$

- Нижний ряд ($i = N - 1$):

$$s_{N-1,j}^{t+1} = f(s_{N-1,j}^t, s_{N-2,j}^t, s_{0,j}^t, s_{N-1,j-1}^t, s_{N-1,j+1}^t)$$

- Левый столбец ($j = 0$):

$$s_{i,0}^{t+1} = f(s_{i,0}^t, s_{i-1,0}^t, s_{i+1,0}^t, s_{i,N-1}^t, s_{i,1}^t)$$

- Правый столбец ($j = N - 1$):

$$s_{i,N-1}^{t+1} = f(s_{i,N-1}^t, s_{i-1,N-1}^t, s_{i+1,N-1}^t, s_{i,N-2}^t, s_{i,0}^t)$$

1.4 Динамическое поведение

Динамическое поведение клеточного автомата описывает эволюцию системы во времени из заданных начальных условий. Характер этой эволюции определяется локальными правилами перехода, но порождает разнообразные глобальные паттерны поведения.

Классификация типов поведения по Вольфраму:

- **Полная сходимость**

Характеризуется быстрым переходом к однородному состоянию. Независимо от начальной конфигурации, система экспоненциально быстро сходится к постоянному состоянию, в котором все клетки принимают одинаковое значение.

- **Сходимость к простым периодическим или статическим паттернам**

Проявляется в возникновении устойчивых или осциллирующих структур. Система эволюционирует к стабильным конфигурациям, которые либо остаются неизменными во времени, либо циклически повторяются с определенным периодом.

- **Сходимость к хаотическому поведению**

Отличается псевдослучайным, неупорядоченным поведением. Малейшие изменения начальных условий приводят к экспоненциальному расхождению траекторий, что делает долгосрочное предсказание невозможным.

- **Сходимость к сложному и случайному поведению**

Порождают сложные, долгоживущие структуры, которые взаимодействуют нелинейным образом. В таких системах отсутствует явно видная закономерность или регулярность.

2 Особенности реализации

2.1 Класс NeumannCellularAutomaton

Класс NeumannCellularAutomaton реализует двумерный клеточный автомат с окрестностью фон Неймана и тороидальными граничными условиями. Класс содержит следующие структуры данных:

- width — целочисленное значение, хранящее ширину квадратного поля
- rule_number — целочисленное значение, хранящее номер правила в десятичной системе счисления
- rule_binary — строка, хранящая правило перехода в 32-битном двоичном формате
- grid — двумерный массив, хранящий состояния всех клеток поля
- initial_state — двумерный массив, сохраняющий начальную конфигурацию поля

Метод `__init__` является конструктором клеточного автомата. Он инициализирует объект класса, преобразует правило в двоичный формат и создает нулевое начальное состояние поля.

Вход: ширина поля, номер правила в десятичной системе счисления.

Выход: инициализированный двумерный клеточный автомат.

Листинг 1. Метод `__init__`

```
1 def __init__(self, width, rule_number):
2     self.width = width
3     self.rule_number = rule_number
4     self.rule_binary = self._convert_rule_to_binary(rule_number)
5     self.grid = np.zeros((width, width), dtype=int)
```

2.1.1 Метод конвертации правила

Метод `_convert_rule_to_binary` выполняет преобразование десятичного номера правила в 32-битную двоичную строку, дополняя результат незначащими нулями слева при необходимости. Это обеспечивает корректное соответствие между числовым представлением правила и таблицей переходов для всех возможных конфигураций окрестности.

Вход: `rule_number` - целочисленное значение номера правила в десятичной системе счисления.

Выход: `binary` - строка, содержащая 32-битное двоичное представление правила.

Листинг 2. Метод `_convert_rule_to_binary`

```
1 def _convert_rule_to_binary(self, rule_number):
2     binary = bin(rule_number)[2:]
3     binary = binary.zfill(32)
4     return binary
```

2.1.2 Метод для определения нового состояния ячейки

Метод `_get_rule_result` реализует механизм применения правила к отдельной ячейке на основе состояний ее соседей. Алгоритм формирует 5-битный индекс из состояний центральной ячейки и четырех соседних клеток, затем использует этот индекс для извлечения соответствующего бита из двоичного представления правила, что определяет новое состояние ячейки.

Вход: `c` - целочисленное состояние центральной ячейки, `n` - целочисленное состояние северной ячейки, `e` - целочисленное состояние восточной ячейки, `s` - целочисленное состояние южной ячейки, `w` - целочисленное состояние западной ячейки.

Выход: целочисленное значение нового состояния центральной ячейки.

Листинг 3. Метод `_get_rule_result`

```
1 def _get_rule_result(self, c, n, s, w, e):
2     index = (c << 4) | (n << 3) | (s << 2) | (w << 1) | e
3     return int(self.rule_binary[index])
```

2.1.3 Метод, реализующий изменение состояния клетки

Метод `set_cell` позволяет установить конкретное состояние для заданной координаты ячейки, выполняя проверку корректности координат перед изменением. Этот метод используется для ручной настройки начальной конфигурации поля и интерактивного редактирования состояния клеток.

Вход: `x` - целочисленная координата ячейки по горизонтали, `y` - целочисленная координата ячейки по вертикали, `state` - целочисленное значение нового состояния ячейки.

Выход: измененное значение клетки.

Листинг 4. Метод `set_cell`

```
1 def set_cell(self, x, y, state):
2     if 0 <= x < self.width and 0 <= y < self.height:
3         self.grid[y, x] = state
```

2.1.4 Метод, реализующий инициализацию клеток поля случайными значениями

Метод `set_initial_conditions_random` инициализирует поле случайными значениями с заданной плотностью заполнения, используя равномерное распределение для генерации состояний клеток.

Вход: `density` - вещественное значение плотности заполнения от 0 до 1, `save_initial` - логическое значение, определяющее необходимость сохранения начального состояния.

Выход: инициализированное поле.

Листинг 5. Метод `set_initial_conditions_random`

```
1 def set_initial_conditions_random(self, density=0.5):
2     self.grid = np.random.choice([0, 1], size=(self.width, self.width),
3                                   p=[1 - density, density])
```

2.1.5 Метод, реализующий очистку поля

Метод `clear` выполняет полную очистку игрового поля, устанавливая всем ячейкам нулевые значения и сбрасывая сохраненное начальное состояние. Данная функция используется для подготовки автомата к новой симуляции с полностью чистым состоянием.

Вход: заполненное поле.

Выход: пустое поле.

Листинг 6. Метод `clear`

```
1 def clear(self):
2     self.grid = np.zeros((self.width, self.width), dtype=int)
```

2.1.6 Метод, реализующий изменение правила

Метод `change_rule` обновляет номер правила и пересчитывает его 32-битное двоичное представление, позволяя использовать новое правило без создания нового экземпляра автомата. Это обеспечивает гибкость в экспериментах с различными правилами переходов.

Вход: `new_rule_number` - целочисленное значение нового номера правила в десятичной системе счисления.

Выход: измененное правило.

Листинг 7. Метод `change_rule`

```
1 def change_rule(self, new_rule_number):
2     self.rule_number = new_rule_number
3     self.rule_binary = self._convert_rule_to_binary(new_rule_number)
```

2.1.7 Метод, реализующий одну итерацию эволюции автомата

Метод `update` выполняет одну итерацию эволюции автомата, применяя правило перехода ко всем ячейкам поля одновременно с учетом тороидальных граничных условий. Для каждой клетки вычисляются состояния соседей, затем применяется правило перехода и формируется новое состояние поля.

Вход: состояние клеток поля, правило.

Выход: двумерный массив, содержащий новое состояние всех ячеек поля после итерации.

Листинг 8. Метод `update`

```
1 def update(self):
2     new_grid = np.zeros((self.width, self.width), dtype=int)
3
4     for i in range(self.width):
5         for j in range(self.width):
6             north = self.grid[(i - 1) % self.width, j]
7             east = self.grid[i, (j + 1) % self.width]
8             south = self.grid[(i + 1) % self.width, j]
9             west = self.grid[i, (j - 1) % self.width]
10            center = self.grid[i, j]
11
12            new_grid[i, j] = self._get_rule_result(center, north, east,
13                                                    south, west)
14
15     self.grid = new_grid
16     return self.grid
```

2.2 Визуальное отображение

Визуализация клеточного автомата реализована с использованием стандартной библиотеки Python — Tkinter, которая предоставляет инструменты для создания графических интерфейсов с элементами управления.

Класс `NeumannCellularAutomatonGUI` реализует интерактивный графический интерфейс для управления клеточным автоматом и визуализации его состояния. Полная реализация класса представлена в приложении.

При запуске программы пользователь видит главное окно, содержащее следующие элементы управления:

1. Поля ввода параметров:

- Поле "Ширина поля" для задания размера квадратной решетки (по умолчанию 30 клеток);

- Поле "Правило" для ввода правила в десятичной системе счисления (по умолчанию 14887125);
- Поле "Итерации" для установки максимального количества шагов эволюции.

2. Панель управления с кнопками:

- "Применить" — подтверждение изменений параметров,
- "Случайно" — заполнение поля случайными состояниями,
- "Очистить" — сброс всех клеток в нулевое состояние,
- "Шаг" — выполнение одной итерации автомата,
- "Запуск" — начало автоматической эволюции,
- "Стоп" — остановка выполнения,
- "Сброс" — возврат к начальному состоянию,
- "График" — построение графика изменения количества живых клеток.

3. Визуальное представление поля реализовано с помощью элемента Canvas, где каждая клетка отображается в виде квадрата. Цвет клетки зависит от ее состояния: черный для живых клеток (1) и белый для мертвых (0). Размер клеток автоматически масштабируется в зависимости от размера окна.

4. Информационная панель отображает:

- Текущий номер итерации,
- Количество живых клеток и общую плотность заполнения,
- Двоичное представление текущего правила.

Для обеспечения корректной работы программы реализованы следующие механизмы:

- Валидация ввода — все поля ввода проверяются на корректность данных, при обнаружении ошибок выводится сообщение через `messagebox`.
- Интерактивное редактирование — пользователь может изменять состояние клеток путем клика левой кнопкой мыши по полю, а также рисовать паттерны с помощью зажатой кнопки мыши.

- Динамическое обновление — метод `update_display` обеспечивает перерисовку поля после каждого изменения, а метод `update_info` обновляет статистическую информацию.
- Автоматическая симуляция — при нажатии кнопки "Запуск" запускается циклическое выполнение итераций с задержкой, что позволяет наблюдать эволюцию системы в реальном времени.
- Визуализация статистики — кнопка "График" запускает построение графика изменения количества живых клеток за 100 шагов, начиная с текущего состояния.

3 Анализ клеточного автомата

В ходе анализа клеточного автомата был проведен ряд экспериментов с автоматом с полем 30 на 30 и количеством итераций 100. Начальные конфигурации поля задавались случайным образом. Анализ эволюции системы показал устойчивую тенденцию к стабилизации количества активных клеток около 200-210.

Полученные данные позволяют классифицировать поведение данного клеточного автомата как соответствующие второму классу по Вольфраму, поскольку система демонстрирует быстрое схождение к периодическому режиму с устойчивыми колебаниями вокруг стабильного уровня заполнения.

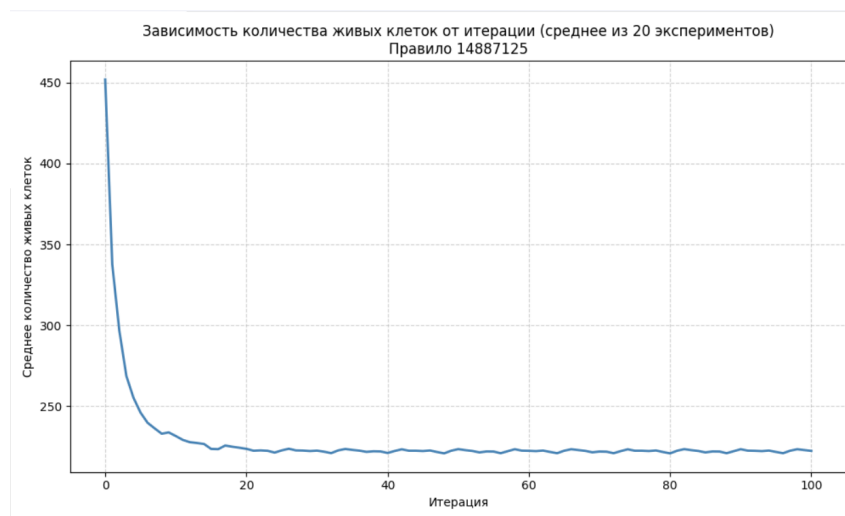


Рис. 2. График изменения количества живых клеток (среднее по 20 экспериментам)

4 Результаты работы программы

При запуске программы пользователь видит стартовое меню (Рис. 3). Далее он может ввести собственные значения для ширины поля, правила и количества итераций, а также задать состояния клеток либо самостоятельно, либо случайным образом (Рис. 4-5).

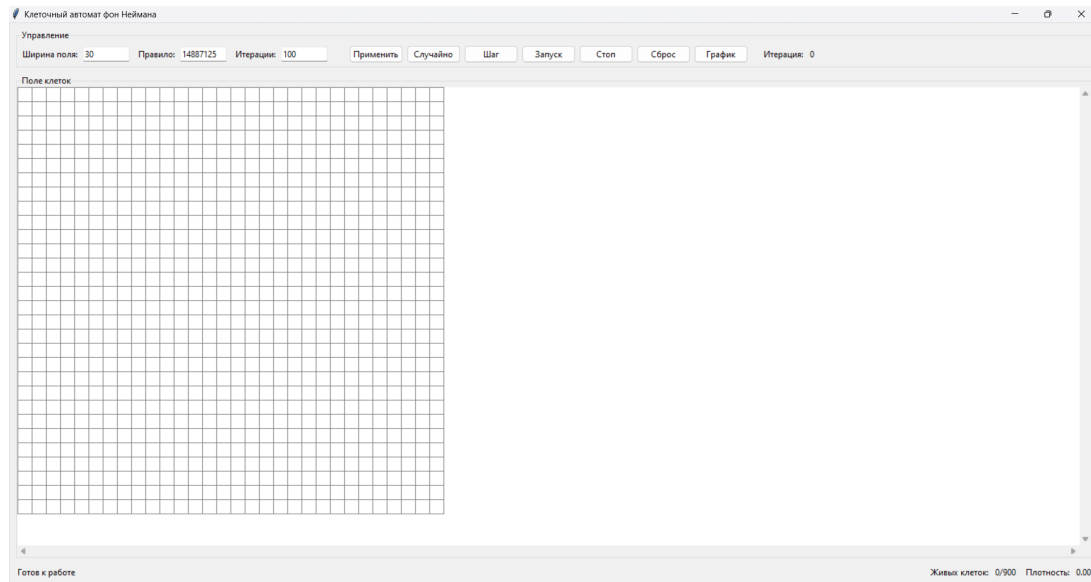


Рис. 3. Меню

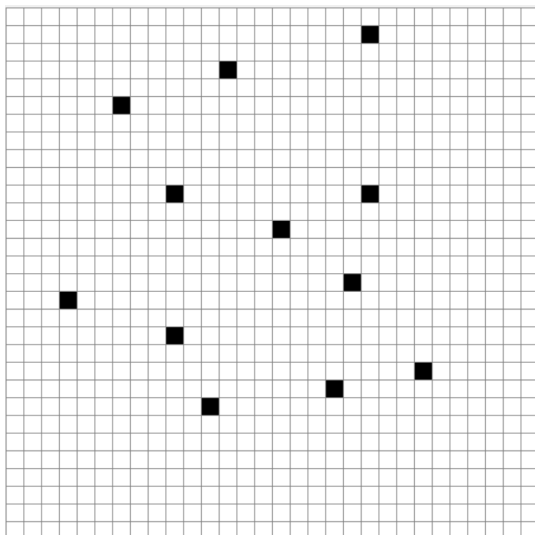


Рис. 4. Ручной ввод

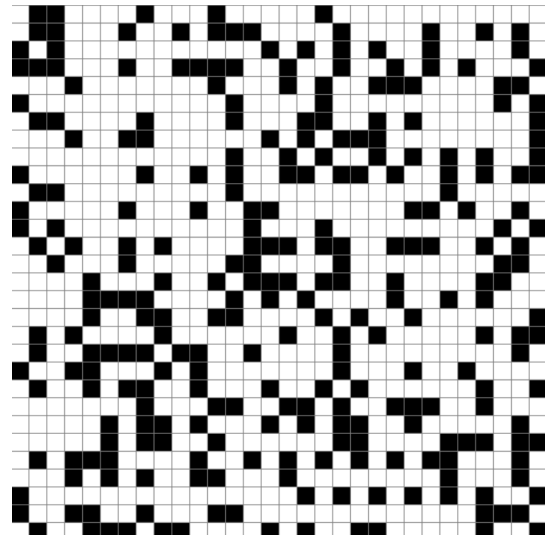


Рис. 5. Случайный ввод

Пример работы программы для простых паттернов представлен на Рис. 6-17.

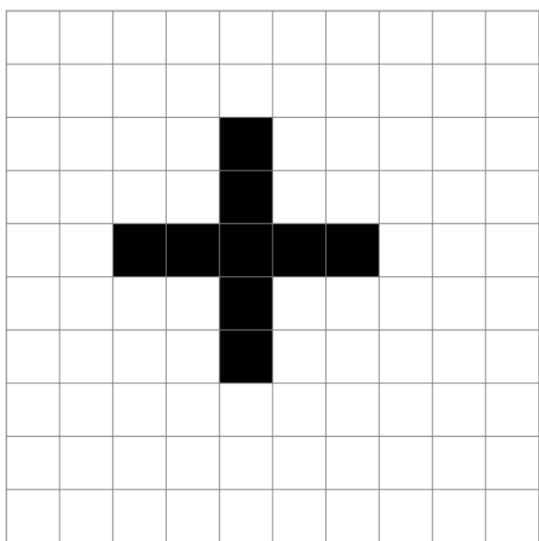


Рис. 6. Шаг 1 (плюс)

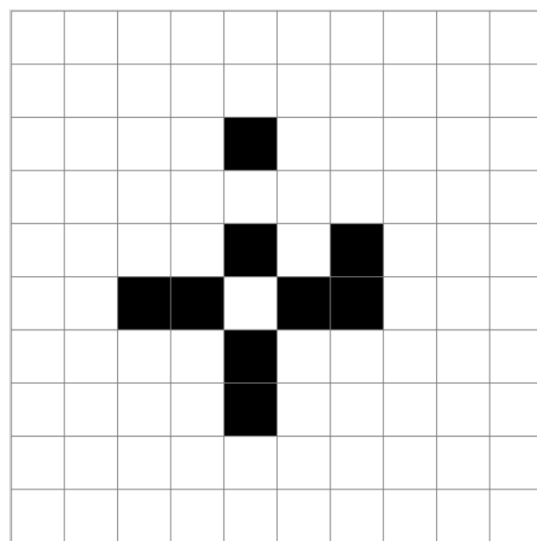


Рис. 7. Шаг 2 (плюс)

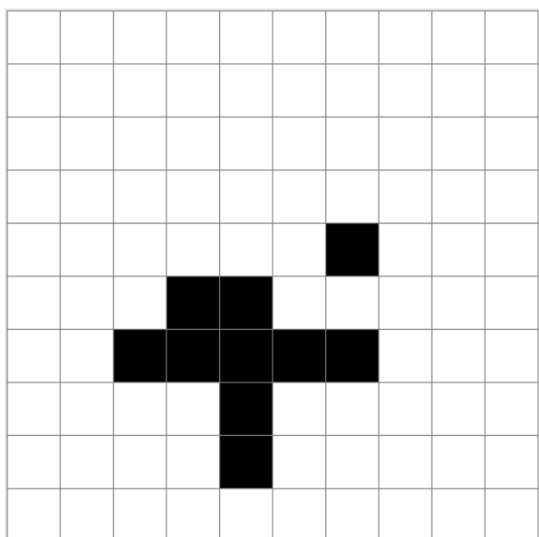


Рис. 8. Шаг 3 (плюс)

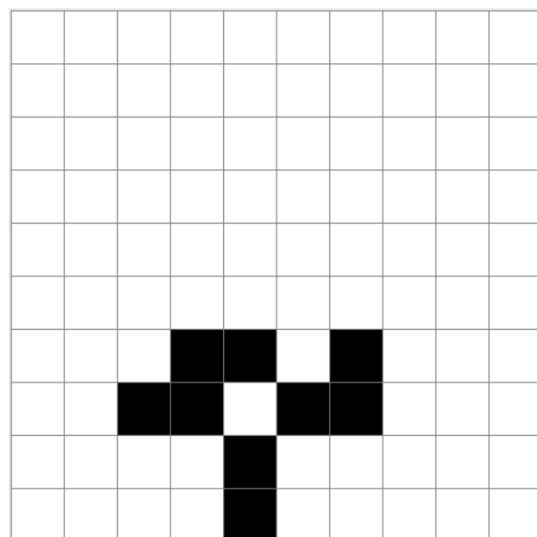


Рис. 9. Шаг 4 (плюс)

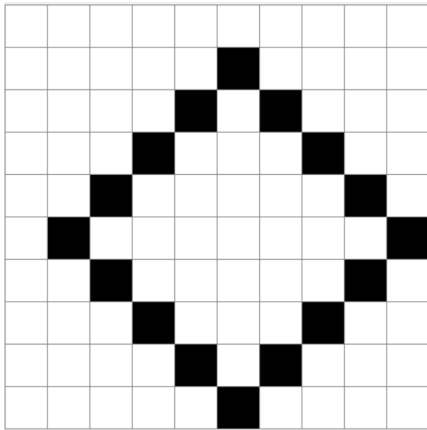


Рис. 10. Шаг 1 (ромб)

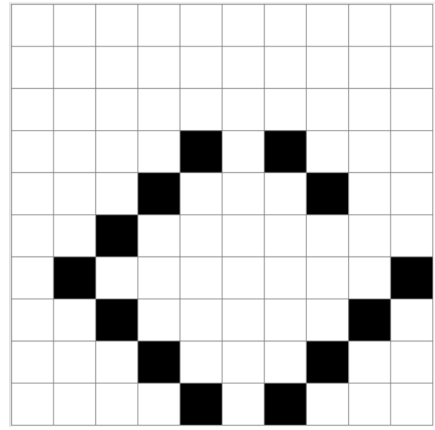


Рис. 11. Шаг 2 (ромб)

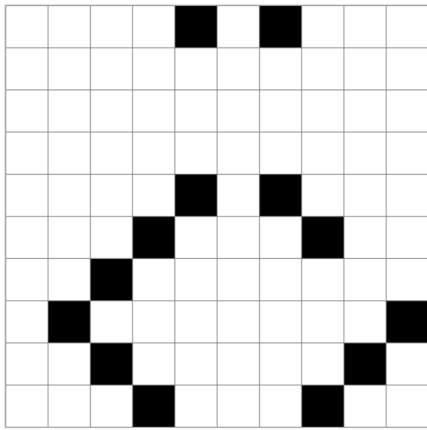


Рис. 12. Шаг 3 (ромб)

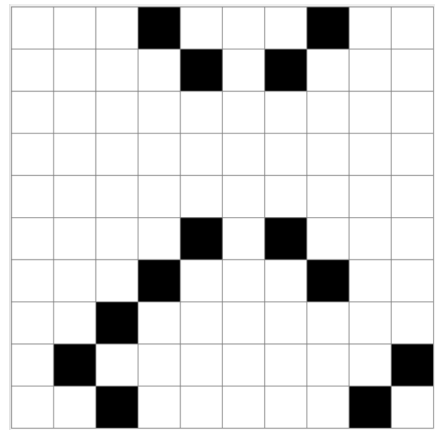


Рис. 13. Шаг 4 (ромб)

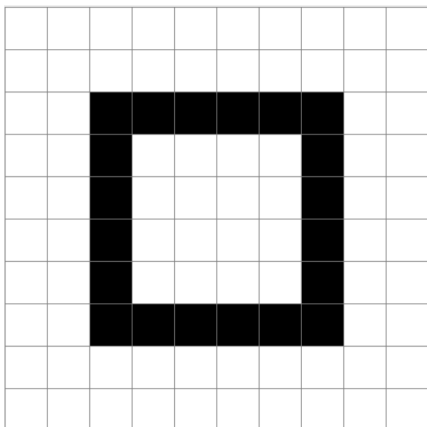


Рис. 14. Шаг 1 (квадрат)

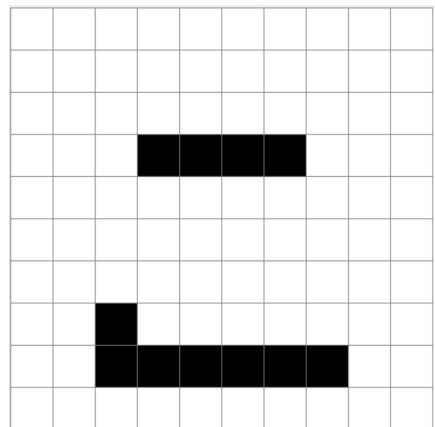


Рис. 15. Шаг 2 (квадрат)

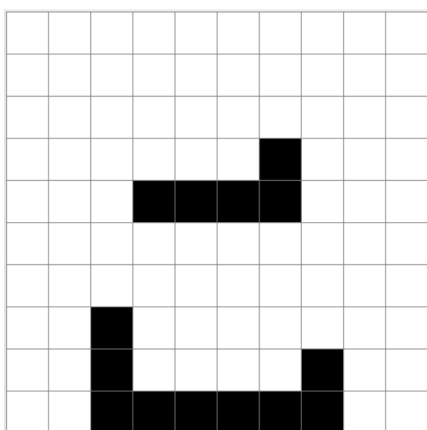


Рис. 16. Шаг 3 (квадрат)

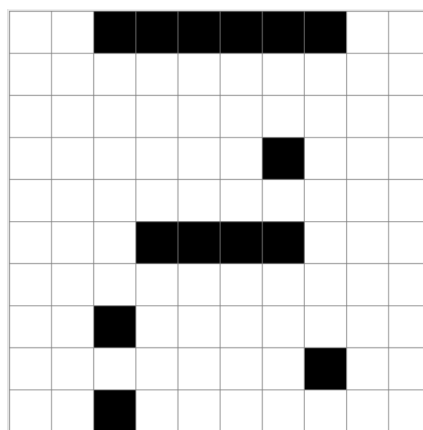


Рис. 17. Шаг 4 (квадрат)

Пример работы программы представлен на Рис. 18–22.

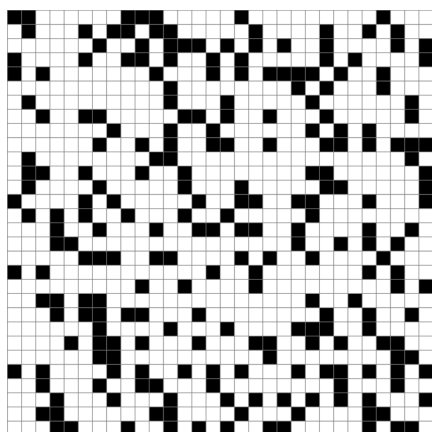


Рис. 18. Шаг 1

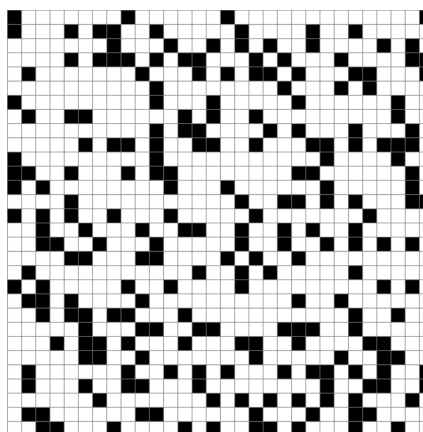


Рис. 19. Шаг 2

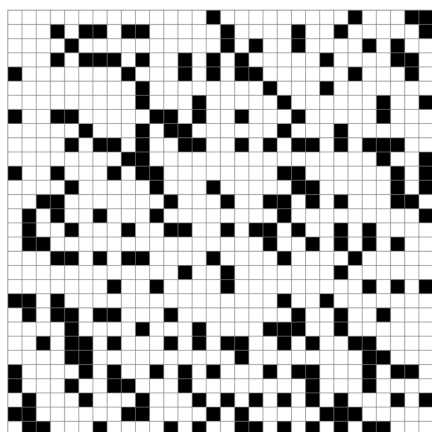


Рис. 20. Шаг 3

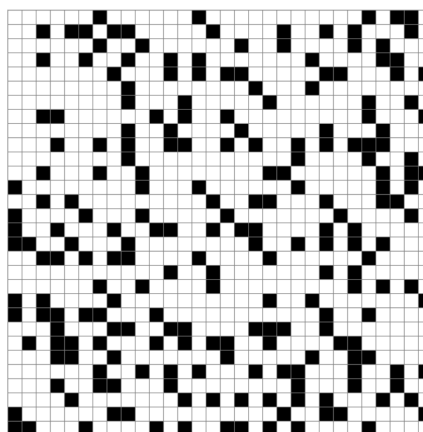


Рис. 21. Шаг 4

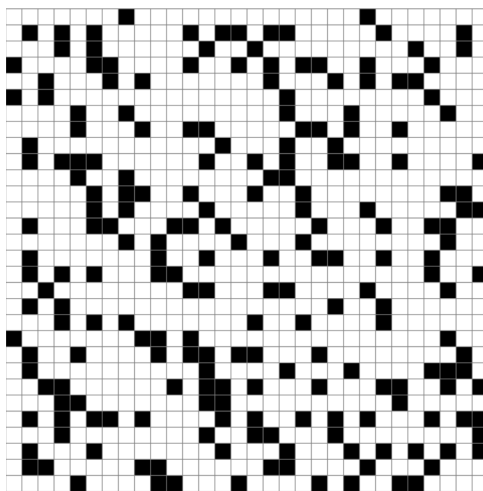


Рис. 22. Шаг 25

При вводе некорректных значений в поля для ввода выводится сообщение об ошибке (Рис. 23).

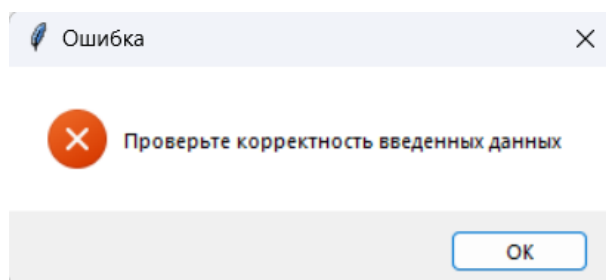


Рис. 23. Некорректный ввод

Заключение

В ходе выполнения лабораторной работы была реализована программа с графическим интерфейсом, которая описывает эволюцию двумерного клеточного автомата с окрестностью фон Неймана, торроидальными граничными условиями и заданным правилом (14887125_{10}). Была реализована возможность задавать размеры поля (до 100×100), правило и количество итераций (до 200) вручную, конфигурацию вручную и случайным образом. Был проведен анализ клеточного автомата, в результате, было выяснено, что он относится ко второму классу по классификации Вольфрама.

Достоинства:

1. Реализован графический интерфейс.
2. Объектно-ориентированный подход позволяет использовать класс в дальнейшем.
3. Есть возможность динамически изменять правило.
4. Есть возможность посмотреть график зависимости количества живых клеток от итерации для любой заданной конфигурации поля.

Недостатки:

1. Отсутствие сохранения графика в файл.
2. Невысокая производительность при больших размерах поля.

Масштабирование:

1. Реализация различных окрестностей (например, окрестности Мура).
2. Оптимизация изменения графического отображения состояния поля (например, использовать алгоритм инкрементальной перерисовки).
3. Добавление возможности задать плотность живых клеток для случайного заполнения поля.

Список литературы

1. Секция «Телематика» – <https://tema.spbstu.ru/algorithm/> (дата обращения – 21.10.2025).
2. Клеточные автоматы. Игра «Жизнь». Часть 1 – <https://habr.com/ru/articles/737672/> (дата обращения – 21.10.2025).
3. Удивительные клеточные автоматы. – <https://habr.com/ru/companies/timeweb/articles/725136/> (дата обращения – 21.10.2025).

Приложение А. Исходный код программы

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import random
4 import tkinter as tk
5 from tkinter import ttk, messagebox
6
7
8 class NeumannCellularAutomaton:
9     def __init__(self, width, rule_number):
10         """
11         Инициализация клеточного автомата
12
13         Args:
14             width: ширина поля (квадратное поле width x width)
15             rule_number: функция
16         """
17         self.width = width
18         self.rule_number = rule_number
19         self.rule_binary = self._convert_rule_to_binary(rule_number)
20         self.grid = np.zeros((width, width), dtype=int)
21
22     def _convert_rule_to_binary(self, rule_number):
23         """Преобразование номера правила в 32-битную двоичную строку"""
24         binary = bin(rule_number)[2:] # Убираем '0b' префикс
25         # Дополняем нулями слева до 32 разрядов
26         binary = binary.zfill(32)
27         return binary
28
29     def _get_rule_result(self, c, n, s, w, e):
30         """
31         Применение правила к ячейке с окрестностью фон Неймана
32
33         Args:
34             c: центральная ячейка
35             n: северная ячейка
36             s: южная ячейка
37             w: западная ячейка
38             e: восточная ячейка
39
40         Returns:
41             Новое состояние ячейки согласно правилу
42         """
43         index = (c << 4) | (n << 3) | (s << 2) | (w << 1) | e
44         return int(self.rule_binary[index])
45
46     def get_grid(self):
47         """Возвращает копию текущего состояния поля"""
48         return self.grid.copy()
49
50     def set_cell(self, x, y, state):
51         """Установка состояния конкретной клетки"""
52         if 0 <= x < self.width and 0 <= y < self.width:
53             self.grid[y, x] = state
54
```

```

55 def get_cell(self, x, y):
56     """Получение состояния конкретной клетки"""
57     if 0 <= x < self.width and 0 <= y < self.width:
58         return self.grid[y, x]
59     return 0
60
61 def set_initial_conditions_random(self, density=0.5):
62     """Случайное заполнение с возможностью сохранения начального со-
63         стояния"""
64     self.grid = np.random.choice([0, 1], size=(self.width, self.
65         width),
66                                   p=[1 - density, density])
67
68 def clear(self):
69     """Полная очистка поля"""
70     self.grid = np.zeros((self.width, self.width), dtype=int)
71
72 def change_rule(self, new_rule_number):
73     """Изменение правила без пересоздания автомата"""
74     self.rule_number = new_rule_number
75     self.rule_binary = self._convert_rule_to_binary(new_rule_number)
76
77 def update(self):
78     """Выполнение одной итерации автомата"""
79     new_grid = np.zeros((self.width, self.width), dtype=int)
80
81     for i in range(self.width):
82         for j in range(self.width):
83             # Топроидальные граничные условия
84             north = self.grid[(i - 1) % self.width, j]
85             east = self.grid[i, (j + 1) % self.width]
86             south = self.grid[(i + 1) % self.width, j]
87             west = self.grid[i, (j - 1) % self.width]
88             center = self.grid[i, j]
89
90             # Применяем правило
91             new_grid[i, j] = self._get_rule_result(center, north,
92                 south, west, east)
93
94     self.grid = new_grid
95     return self.grid
96
97 def get_statistics(self):
98     """Получение статистики по текущему состоянию"""
99     alive_cells = np.sum(self.grid)
100     total_cells = self.width * self.width
101     density = alive_cells / total_cells
102
103     return {
104         'alive_cells': alive_cells,
105         'total_cells': total_cells,
106         'density': density
107     }
108
109 class NeumannCellularAutomatonGUI:

```

```

108     def __init__(self, root):
109         self.root = root
110         self.root.title("Клеточный автомат фон Неймана")
111         self.root.geometry("800x600")
112
113         # Переменные для хранения состояния
114         self.width = 30
115         self.rule_number = 14_887_125
116         self.max_iterations = 100
117         self.current_iteration = 0
118         self.is_running = False
119         self.automaton = None
120
121         self.create_widgets()
122         self.initialize_automaton()
123
124     def create_widgets(self):
125         """Создание элементов интерфейса"""
126         # Основной фрейм
127         main_frame = ttk.Frame(self.root, padding="10")
128         main_frame.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S)
129             )
130
131         # Настройка весов для растягивания
132         self.root.columnconfigure(0, weight=1)
133         self.root.rowconfigure(0, weight=1)
134         main_frame.columnconfigure(1, weight=1)
135         main_frame.rowconfigure(1, weight=1)
136
137         # Фрейм управления
138         control_frame = ttk.LabelFrame(main_frame, text="Управление",
139             padding="5")
140         control_frame.grid(row=0, column=0, colspan=2, sticky=(tk.W,
141             tk.E), pady=(0, 10))
142
143         # Поля ввода
144         ttk.Label(control_frame, text="Ширина поля:").grid(row=0, column
145             =0, sticky=tk.W, padx=(0, 5))
146         self.width_var = tk.StringVar(value=str(self.width))
147         width_entry = ttk.Entry(control_frame, textvariable=self.
148             width_var, width=10)
149         width_entry.grid(row=0, column=1, sticky=tk.W, padx=(0, 10))
150
151         ttk.Label(control_frame, text="Правило:").grid(row=0, column=2,
152             sticky=tk.W, padx=(0, 5))
153         self.rule_var = tk.StringVar(value=str(self.rule_number))
154         rule_entry = ttk.Entry(control_frame, textvariable=self.rule_var
155             , width=10)
156         rule_entry.grid(row=0, column=3, sticky=tk.W, padx=(0, 10))
157
158         ttk.Label(control_frame, text="Итерации:").grid(row=0, column=4,
159             sticky=tk.W, padx=(0, 5))
160         self.iter_var = tk.StringVar(value=str(self.max_iterations))
161         iter_entry = ttk.Entry(control_frame, textvariable=self.iter_var
162             , width=10)
163         iter_entry.grid(row=0, column=5, sticky=tk.W, padx=(0, 10))

```

```

155
156 # Кнопки управления
157 button_frame = ttk.Frame(control_frame)
158 button_frame.grid(row=0, column=6, sticky=tk.W, padx=(20, 0))
159
160 ttk.Button(button_frame, text="Применить", command=self.
161         apply_settings).pack(side=tk.LEFT, padx=(0, 5))
162 ttk.Button(button_frame, text="Случайно", command=self.
163         random_fill).pack(side=tk.LEFT, padx=(0, 5))
164 ttk.Button(button_frame, text="Шаг", command=self.step).pack(
165         side=tk.LEFT, padx=(0, 5))
166 ttk.Button(button_frame, text="Запуск", command=self.start).pack(
167         side=tk.LEFT, padx=(0, 5))
168 ttk.Button(button_frame, text="Стоп", command=self.stop).pack(
169         side=tk.LEFT)
170 ttk.Button(button_frame, text="Сброс", command=self.clear_grid).
171         pack(side=tk.LEFT, padx=(5, 0))
172 ttk.Button(button_frame, text="График", command=self.show_graph)
173         .pack(side=tk.LEFT, padx=(5, 0))
174
175 # Счетчик итераций
176 info_frame = ttk.Frame(control_frame)
177 info_frame.grid(row=0, column=7, sticky=tk.E, padx=(20, 0))
178
179 ttk.Label(info_frame, text="Итерация:").pack(side=tk.LEFT)
180 self.iteration_label = ttk.Label(info_frame, text="0")
181 self.iteration_label.pack(side=tk.LEFT, padx=(5, 0))
182
183 # Фрейм для поля клеток
184 grid_frame = ttk.LabelFrame(main_frame, text="Поле клеток")
185 grid_frame.grid(row=1, column=0, columnspan=2, sticky=(tk.W, tk.
186         E, tk.N, tk.S), pady=(0, 10))
187 grid_frame.columnconfigure(0, weight=1)
188 grid_frame.rowconfigure(0, weight=1)
189
190 # Холст для отрисовки клеток
191 self.canvas = tk.Canvas(grid_frame, bg='white', borderwidth=0,
192         highlightthickness=0)
193 self.canvas.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S
194         ))
195
196 # Привязка событий мыши
197 self.canvas.bind("<Button-1>", self.on_canvas_click)
198 self.canvas.bind("<B1-Motion>", self.on_canvas_drag)
199
200 # Полоса прокрутки
201 v_scrollbar = ttk.Scrollbar(grid_frame, orient=tk.VERTICAL,
202         command=self.canvas.yview)
203 v_scrollbar.grid(row=0, column=1, sticky=(tk.N, tk.S))
204 h_scrollbar = ttk.Scrollbar(grid_frame, orient=tk.HORIZONTAL,
205         command=self.canvas.xview)
206 h_scrollbar.grid(row=1, column=0, sticky=(tk.W, tk.E))
207
208 self.canvas.configure(yscrollcommand=v_scrollbar.set,
209         xscrollcommand=h_scrollbar.set)
210

```

```

198     # Информационная панель
199     info_panel = ttk.Frame(main_frame)
200     info_panel.grid(row=2, column=0, columnspan=2, sticky=(tk.W, tk.
        E))
201
202     self.info_label = ttk.Label(info_panel, text="Готов к работе")
203     self.info_label.pack(side=tk.LEFT)
204
205     # Статистика
206     stats_frame = ttk.Frame(info_panel)
207     stats_frame.pack(side=tk.RIGHT)
208
209     ttk.Label(stats_frame, text="Живых клеток:").pack(side=tk.LEFT)
210     self.live_cells_label = ttk.Label(stats_frame, text="0")
211     self.live_cells_label.pack(side=tk.LEFT, padx=(5, 10))
212
213     ttk.Label(stats_frame, text="Плотность:").pack(side=tk.LEFT)
214     self.density_label = ttk.Label(stats_frame, text="0.00")
215     self.density_label.pack(side=tk.LEFT, padx=(5, 0))
216
217     def initialize_automaton(self):
218         """Инициализация автомата"""
219         try:
220             new_width = int(self.width_var.get())
221             new_rule = int(self.rule_var.get())
222             self.max_iterations = int(self.iter_var.get())
223
224             # Если размер изменился или автомата еще нет
225             if not self.automaton or new_width != self.width or new_rule
                != self.rule_number:
226                 self.width = new_width
227                 self.rule_number = new_rule
228                 self.automaton = NeumannCellularAutomaton(self.width,
                    self.rule_number)
229             else:
230                 # Только меняем правило
231                 self.automaton.change_rule(new_rule)
232
233             self.current_iteration = 0
234             self.update_display()
235             self.update_info()
236
237         except ValueError as e:
238             messagebox.showerror("Ошибка", "Проверьте корректность введе
                нных данных")
239
240     def apply_settings(self):
241         """Применение новых настроек"""
242         self.initialize_automaton()
243
244     def random_fill(self):
245         """Случайное заполнение поля"""
246         if self.automaton:
247             self.automaton.set_initial_conditions_random(0.3)
248             self.current_iteration = 0
249             self.update_display()

```

```

250         self.update_info()
251
252     def clear_grid(self):
253         """Очистка поля"""
254         if self.automaton:
255             self.automaton.clear()
256             self.current_iteration = 0
257             self.update_display()
258             self.update_info()
259
260     def step(self):
261         """Выполнение одного шага"""
262         if self.automaton and self.current_iteration < self.
            max_iterations:
263             self.automaton.update()
264             self.current_iteration += 1
265             self.update_display()
266             self.update_info()
267
268     def start(self):
269         """Запуск автоматического выполнения"""
270         if not self.automaton:
271             return
272
273         self.is_running = True
274         self.run_simulation()
275
276     def stop(self):
277         """Остановка выполнения"""
278         self.is_running = False
279
280     def run_simulation(self):
281         """Выполнение симуляции"""
282         if self.is_running and self.current_iteration < self.
            max_iterations:
283             self.step()
284             self.root.after(100, self.run_simulation) # 100ms задержка
285         else:
286             self.is_running = False
287
288     def on_canvas_click(self, event):
289         """Обработка клика по клетке"""
290         if not self.automaton:
291             return
292
293         # Получаем координаты клетки
294         cell_size = self.get_cell_size()
295         x = int(event.x // cell_size)
296         y = int(event.y // cell_size)
297
298         if 0 <= x < self.width and 0 <= y < self.height:
299             # Меняем состояние клетки через метод автомата
300             current_state = self.automaton.get_cell(x, y)
301             self.automaton.set_cell(x, y, 1 - current_state)
302             self.current_iteration = 0 # Сбрасываем счетчик при ручном
                изменении

```

```

303         self.update_display()
304         self.update_info()
305
306     def on_canvas_drag(self, event):
307         """Обработка перетаскивания для рисования"""
308         self.on_canvas_click(event)
309
310     def get_cell_size(self):
311         """Вычисление размера клетки для отображения"""
312         canvas_width = self.canvas.winfo_width()
313         canvas_height = self.canvas.winfo_height()
314
315         if canvas_width > 1 and canvas_height > 1:
316             cell_size = min(canvas_width // self.width, canvas_height //
317                             self.width)
318             return max(5, cell_size) # Минимальный размер 5px
319         return 20 # Размер по умолчанию
320
321     def update_display(self):
322         """Обновление отображения поля"""
323         if not self.automaton:
324             return
325
326         self.canvas.delete("all")
327
328         cell_size = self.get_cell_size()
329         total_size = cell_size * self.width
330
331         # Устанавливаем область прокрутки
332         self.canvas.configure(scrollregion=(0, 0, total_size, total_size))
333
334         # Отрисовка клеток
335         for y in range(self.width):
336             for x in range(self.width):
337                 # Используем метод get_cell вместо прямого доступа к
338                 # grid
339                 cell_state = self.automaton.get_cell(x, y)
340                 color = 'black' if cell_state == 1 else 'white'
341                 x1 = x * cell_size
342                 y1 = y * cell_size
343                 x2 = x1 + cell_size
344                 y2 = y1 + cell_size
345
346                 self.canvas.create_rectangle(x1, y1, x2, y2, fill=color,
347                                              outline='gray', width=1)
348
349     def update_info(self):
350         """Обновление информации"""
351         self.iteration_label.config(text=str(self.current_iteration))
352
353         if self.automaton:
354             stats = self.automaton.get_statistics()
355             self.live_cells_label.config(text=f"{stats['alive_cells']}/{stats['total_cells']}")
356             self.density_label.config(text=f"{stats['density']:.2f}")

```

```

354         else:
355             self.info_label.config(text="Автомат не инициализирован")
356
357     def show_graph(self):
358         """Показать график изменения количества живых клеток"""
359         try:
360             if not self.automaton:
361                 messagebox.showwarning("Предупреждение", "Сначала инициализируйте автомат")
362                 return
363
364             # Используем текущий автомат, но сохраняем состояние
365             original_grid = self.automaton.get_grid().copy()
366             current_iteration = self.current_iteration
367
368             # Собираем историю будущих состояний
369             live_history = []
370             steps = 100
371
372             for i in range(steps):
373                 stats = self.automaton.get_statistics()
374                 live_history.append(stats['alive_cells'])
375                 self.automaton.update()
376
377             # Восстанавливаем исходное состояние
378             self.automaton = NeumannCellularAutomaton(self.width, self.rule_number)
379             for y in range(self.width):
380                 for x in range(self.width):
381                     self.automaton.set_cell(x, y, original_grid[y, x])
382             self.current_iteration = current_iteration
383
384             # Строим график
385             plt.figure(figsize=(10, 6))
386             plt.plot(live_history, 'b-', linewidth=2)
387             plt.title(f'Эволюции (Правило {self.rule_number})')
388             plt.xlabel('Итерации')
389             plt.ylabel('Живые клетки')
390             plt.grid(True, alpha=0.3)
391
392             plt.axhline(y=live_history[0], color='red', linestyle='--',
393                        label=f'Текущее: {live_history[0]} клеток')
394             plt.legend()
395             plt.tight_layout()
396             plt.show()
397
398         except Exception as e:
399             messagebox.showerror("Ошибка", f"Ошибка: {str(e)}")
400
401 if __name__ == "__main__":
402     root = tk.Tk()
403     app = NeumannCellularAutomatonGUI(root)
404     root.mainloop()

```